Lista de Exercícios – Polimorfismo

Parte 1 – Questões teóricas

Questão 1. Classifique cada situação como **sobrecarga (SC)** ou **sobrescrita (SS)** e justifique em uma linha.

1. A classe `Conta` tem `depositar(double valor)` e `depositar(double valor, String descricao)`.
2. A classe `ContaPoupanca`, subclasse de `Conta`, redefine `sacar(double valor)` para cobrar uma taxa.
3. A classe `Produto` implementa `public String toString()`.
4. A classe `Produto` implementa `public boolean equals(Produto p)`.
5. A classe `Impressora` tem `imprimir(String texto)` e `imprimir(int numero)`.

Questão 2. Marque V (verdadeiro) ou F (falso). Corrija as falsas.

1. ( ) A anotação `@Override` é obrigatória para que a sobrescrita funcione.
2. ( ) Na sobrescrita, quem decide qual método será executado é o tipo da variável.
3. ( ) O upcasting (subclasse vista como superclasse) é automático e não precisa de cast.
4. ( ) Um downcasting errado, como `(Cachorro) umGato`, gera erro de compilação.
5. ( ) Toda classe Java herda de `Object`, mesmo sem escrever `extends`.
6. ( ) Um atributo `final` pode receber valor no construtor, mas não em um método `set`.
7. ( ) Os métodos de uma classe `final` podem ser sobrescritos pelas suas subclasses.

Questão 3. Responda com suas palavras (até 3 linhas cada).

1. Na analogia do controle remoto, o que o **tipo da variável** define e o que o **objeto** define?
2. Qual a diferença entre `==` e `equals()` ao comparar objetos? Por que comparamos Strings com `equals()`?
3. Um sistema percorre um vetor `Animal[]` chamando `fazerSom()`. Por que criar a classe `Pato` não exige mudar esse laço?
4. Qual a diferença entre `super.fazerSom()` dentro de um método e `super()` dentro de um construtor?

Parte 2 – Teste de mesa

Considere as classes abaixo.

```
public class Veiculo {
    public void mover() {
        System.out.println("Veículo se movendo");
    }
    public String descricao() {
        return "Veículo";
```

```
    }
}

public class Carro extends Veiculo {
    @Override
    public void mover() {
        System.out.println("Carro rodando");
    }
    public void buzinar() {
        System.out.println("Bi-bi!");
    }
}

public class Moto extends Veiculo {
    @Override
    public void mover() {
        super.mover();
        System.out.println("Moto acelerando");
    }
}

public class Bicicleta extends Veiculo {
    @Override
    public String descricao() {
        return "Bicicleta";
    }
}
```

Questão 4. Para cada trecho, escreva a saída. Se não compilar ou der erro ao executar, diga qual e por quê.

a)

```
Veiculo v = new Carro();
v.mover();
```

b)

```
Veiculo v = new Moto();
v.mover();
```

c)

```
Veiculo v = new Bicicleta();
v.mover();
System.out.println(v.descricao());
```

d)

```
Veiculo v = new Carro();
v.buzinar();
```

e)

```
Veiculo v = new Moto();
Carro c = (Carro) v;
c.buzinar();
```

f)

```
Veiculo v = new Carro();
System.out.println(v instanceof Veiculo);
System.out.println(v instanceof Moto);
System.out.println(v instanceof Object);
```

g)

```
Object o = new Moto();
o.mover();
```

h)

```
Veiculo[] frota = new Veiculo[3];
frota[0] = new Carro();
frota[1] = new Bicicleta();
frota[2] = new Moto();

for (int i = 0; i < frota.length; i++) {
    System.out.println(frota[i].descricao());
    if (frota[i] instanceof Carro) {
        ((Carro) frota[i]).buzinar();
    }
}
```

Parte 3 – Encontre o erro

Questão 5. Um colega escreveu o `equals()` da classe `Aluno` assim:

```
public class Aluno {
    private int matricula;

    public Aluno(int matricula) {
        this.matricula = matricula;
    }
    public int getMatricula() {
        return matricula;
    }
    public boolean equals(Aluno outro) {
        return this.matricula == outro.getMatricula();
    }
}
```

```
Aluno a1 = new Aluno(123);
Aluno a2 = new Aluno(123);
Object o = a2;

System.out.println(a1 == a2);
System.out.println(a1.equals(a2));
System.out.println(a1.equals(o));
```

1. Qual é a saída?
2. O método escrito é sobrecarga ou sobrescrita? Por quê?
3. Reescreva o método corretamente, usando `@Override`, `instanceof` e cast.

Questão 6. O gerente reclamou que está recebendo o mesmo salário dos demais funcionários.

```
public class Funcionario {
    public double calcularSalario() {
        return 2000.0;
    }
}

public class Gerente extends Funcionario {
    public double calcularSalaro() {
        return 2000.0 * 1.5;
    }
}
```

```
Funcionario f = new Gerente();
System.out.println(f.calcularSalario());
```

1. O que é impresso? Explique o motivo.
2. Como a anotação `@Override` teria evitado esse problema?
3. Reescreva `calcularSalario()` em `Gerente` usando `super.calcularSalario()`.

Questão 7. Indique quais linhas **não compilam** e explique o motivo de cada uma.

```
public final class Moeda {                    // L1
    private final String simbolo;             // L2

    public Moeda(String simbolo) {
        this.simbolo = simbolo;               // L3
    }
    public void setSimbolo(String s) {
        this.simbolo = s;                     // L4
    }
    public final String getSimbolo() {        // L5
        return simbolo;
    }
}

public class Real extends Moeda {             // L6
}

public class Conta {
    public final void exibirSaldo() { }       // L7
}

public class ContaCorrente extends Conta {
    @Override
    public void exibirSaldo() { }             // L8
}
```

Parte 4 – Implementação (no papel)

Questão 8. Formas geométricas

1. Crie a classe `Forma` com o atributo `nome` (String), um construtor e o método `double area()`, que retorna 0.

2. Sobrescreva `toString()` em `Forma` para retornar, por exemplo, `"Retângulo - área: 12.0"`.
3. Crie `Retangulo` (base e altura) e `Circulo` (raio), sobrescrevendo `area()` com `@Override`.
4. No `main`, crie um vetor `Forma[]` com 3 formas, imprima cada uma com `println` e mostre a soma das áreas.

Questão 9. Formas de pagamento

1. Crie a classe `Pagamento` com o atributo `valor` e o método `double calcularTotal()`, que retorna o próprio valor.
2. `PagamentoPix` sobrescreve `calcularTotal()` com 5% de desconto.
3. `PagamentoCartao` tem o atributo `parcelas` (int). Sobrescreve `calcularTotal()` com acréscimo de 10% quando houver mais de 1 parcela.
4. Só `PagamentoCartao` tem o método `exibirParcelas()`, que imprime, por exemplo, `"3x de R$ 36.67"`.
5. Crie a classe `Caixa` com o método `processar(Pagamento p)`: ele imprime o total e, se o pagamento for com cartão, chama também `exibirParcelas()`.

Questão 10. Livros da biblioteca

1. Crie a classe `Livro` com os atributos `isbn` (String, `final`) e `titulo`.
2. Sobrescreva `toString()` para retornar `"isbn - titulo"`.
3. Sobrescreva `equals(Object obj)`: dois livros são iguais quando têm o mesmo ISBN.
4. Escreva a saída do trecho abaixo, já com sua classe pronta.

```
Livro l1 = new Livro("978-85", "Java: como programar");
Livro l2 = new Livro("978-85", "Java: como programar (4. ed.)");
Livro l3 = l1;

System.out.println(l1 == l2);
System.out.println(l1.equals(l2));
System.out.println(l1 == l3);
System.out.println(l2);
```

Desafio. Se o método `area()` de `Forma` fosse declarado `final`, o que aconteceria com `Retangulo` e `Circulo`? E se a classe `Forma` fosse `final`?